Multi-Robot Inspection: Simulation and Semantic Planning

PhD Assignment Report – Ref. no. 4646-2026

Akshit Sharma
Application for the PhD position in Robotics and Artificial Intelligence
October 2026

Code and evidence: github.com/Akshit-Sh/ltu-robotics-ai-phd-assignment

Summary. This report covers all four assignment tasks. **(1)** I built a three-robot ROS 2 Humble / Gazebo Classic inspection testbed with namespaced sensors, SLAM Toolbox mapping, a centralized ten-waypoint mission executor, a custom A* planner with a LiDAR stop/resume guard, and a failure-detection and waypoint-reassignment mechanism. **(2)** I built a language-grounded semantic planning pipeline: a configured scene graph, a local Qwen 2.5 1.5B planner, a plan validator and event-driven replanning. **(3)** I analyse long-term deployment using the literature. **(4)** I propose a research framework for evidence-aware semantic planning and task allocation when the environment changes over time.

Implemented work, recorded results, literature analysis and proposed research are labelled separately throughout. Section 3 collects the limits of the current work and the next steps.

Contents

1 Introduction **2**

2 Assignment Tasks **2**
2.1 Multi-Robot Simulation, Sensing, Mapping and Navigation . . . 2
2.2 Language-Grounded Semantic Planning and Event Handling . . . 5
2.3 Long-Term Deployment Challenges . . . 7
2.4 Open Research Problem and Proposed Approach . . . 8

3 Limitations and Future Work **12**
3.1 Navigation and mapping . . . 12
3.2 Semantic planning . . . 12
3.3 Long-term deployment . . . 12

4 Conclusion **12**

5 Reproducibility **13**

6 Time Spent and AI Assistance Declaration **13**

References **13**

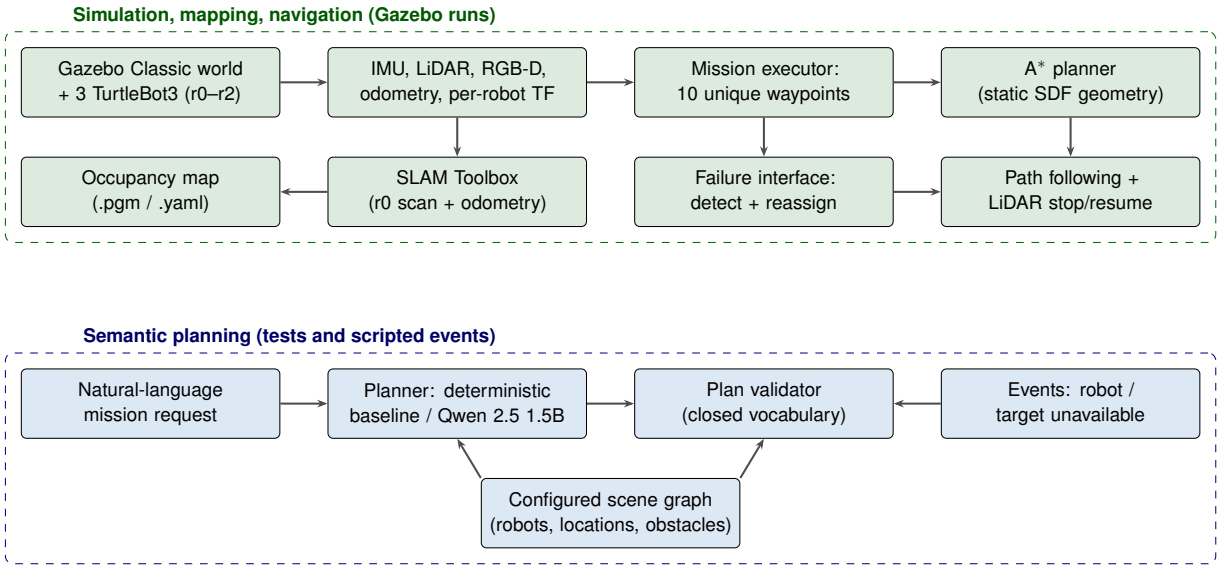


Figure 1. System architecture. The upper pipeline (green) ran in Gazebo. The lower pipeline (blue) is a separate software pipeline over configured scene knowledge, tested with automated tests and scripted events. Arrows show the implemented data flow within each pipeline.

1. Introduction

This project builds a multi-robot inspection testbed on ROS 2 Humble and Gazebo Classic. I use it to study how a team of robots can receive, carry out and revise inspection tasks. The report follows the four assignment tasks (Figure 1): a simulated three-robot team with mapping, navigation and mission planning (Section 2.1); a language-grounded semantic planner with validation and event handling (Section 2.2); an analysis of long-term deployment (Section 2.3); and a research problem that follows from the implementation (Section 2.4).

Contributions. (1) A reproducible three-robot Gazebo environment with robot-specific namespaces, TF frames, and IMU, 2D LiDAR and RGB-D sensing. (2) A navigation pipeline with centralized waypoint allocation, a custom A* planner over static Gazebo SDF geometry, odometry-based path following and a LiDAR stop/resume guard. In the recorded A*-enabled run it completed all ten unique waypoints. (3) Software-level failure detection that reassigns unfinished waypoints. (4) A semantic planning pipeline (configured scene graph, deterministic baseline, local language-model planner, validator and event handler) with a recorded regression run of 61 tests. (5) A research direction that follows from this work: evidence-aware task validation and task ownership for multi-robot teams in changing environments.

How to read this report. The table below shows what kind of content each section holds.

Section	Kind of content	Evidence
2.1, 2.2	Implemented work, recorded experiments	Gazebo runs, logs, scripted demonstrations, automated tests
2.3	Literature-based analysis	Cited papers; no experiments
2.4	Proposed research	Formulation and evaluation plan; no results

2. Assignment Tasks

2.1 Multi-Robot Simulation, Sensing, Mapping and Navigation

Platform and robots. The testbed runs on Ubuntu 22.04 (WSL2) with ROS 2 Humble and Gazebo Classic. Three TurtleBot3 Burger robots, r0, r1 and r2, start at $(0,0)$, $(2,0)$ and $(0,2)$ m in a shared world, `inspection_world.world`. The world has boundary walls and two static boxes (`block_a`, `block_b`). Figure 3(a) shows the robots with their LiDAR rays.

Each robot gets its own namespace, which keeps its sensor and control topics separate: `/rX/cmd_vel`, `/rX/scan`, `/rX/imu` and `/rX/odom`. A separate `robot_state_publisher` per robot adds the frame prefix `rX/`. This lets all robots publish on the shared `/tf` and `/tf_static` topics without frame-name clashes. Each robot carries an IMU, a 2D LiDAR and an RGB-D camera, configured as in Table 2.

Table 1. Key outcomes from the recorded runs (values from repository evidence).

Component	Recorded outcome
Simulation and TF	Three TurtleBot3 Burger robots (r0, r1, r2). Each has its own `odom`, `base_footprint`, `base_link`, camera, IMU, LiDAR and caster frames (Figure 2).
Nominal mission (Run A, 18 Oct 2026, A*)	`MISSION_COMPLETE`: all ten unique waypoints (wp01–wp10) completed in 197.4 s from assignment. Final goal distances 0.048–0.050 m against a 0.15 m tolerance. Four LiDAR stop/clear event pairs logged.
Mapping	SLAM Toolbox map, 152 × 160 cells at 0.05 m (7.60 × 8.00 m); 63.5% of cells known (60.1% free, 3.4% occupied).
Failure injection (separate run)	`ROBOT_FAILED` for r0 logged 8.3 s after assignment. Its four unfinished waypoints were redistributed, and r2 then reached wp06 and wp09.
Semantic pipeline	Scene-graph hierarchy validation, structured planning with Qwen 2.5 1.5B, plan validation, two scripted event types. Recorded regression run: 61 collected, 60 passed, 1 skipped, 0 failed.

Table 2. Configured sensor settings in the robot model files.

Sensor	Configured settings
IMU	200 Hz update rate
2D LiDAR	10 Hz; 360 samples; 0.12–3.5 m range
RGB-D camera	10 Hz; 320 × 240; 1.047 rad horizontal FOV; 0.10–8.0 m depth range

Sensor and frame checks. Figure 2 shows the recorded TF tree. For each robot the chain is `rX/odom` → `rX/base_footprint` → `rX/base_link`, with the child frames `camera_link`, `caster_back_link`, `imu_link` and `base_scan`. `/tf_static` has three publishers, one per namespace. I checked the scan, RGB, depth and odometry topics of all three robots. The RGB and depth headers carry the expected robot-specific camera frame ID.

Short WSL2 measurements gave RGB rates of about 6.7 Hz (r0) and 7.0 Hz (r1) and depth rates of 6.5–7.2 Hz. After the sensor update the LiDAR ran at about 8.7 Hz, against 10 Hz configured (it was 3.2–4.0 Hz before). Gazebo ran with software rendering at a real-time factor of 0.67–0.89, so wall-clock times in the logs are not simulated time.

Mapping. SLAM Toolbox runs in mapping mode (`config/slam_r0.yaml`) on `/r0/scan`, using the frames `r0/odom`, `r0/base_footprint` and `map`, at 0.05 m resolution with simulated time. The saved map (`maps/inspection_map.pgm` and `.yaml`, Figure 3b) has 152 × 160 cells over a 7.60 × 8.00 m extent: 60.1% free, 3.4% occupied and 36.5% unknown, so 63.5% is known. It shows the arena walls, the box outlines and the scanned interior.

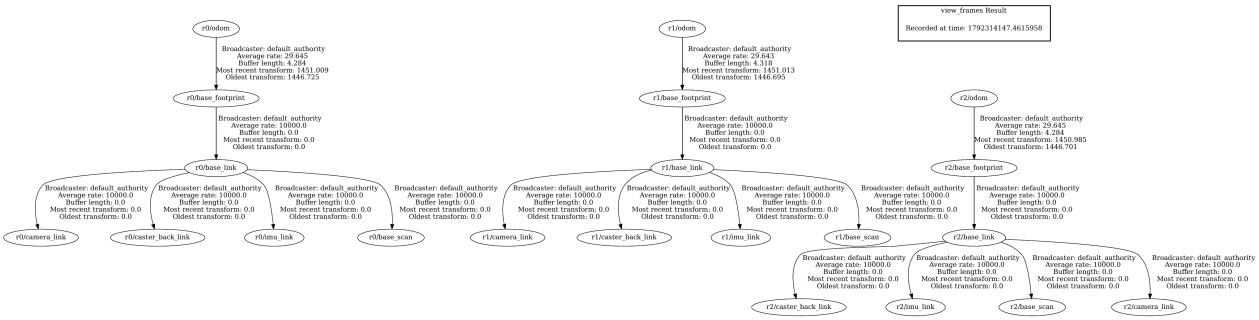
Centralized mission planning. A `mission_executor` node allocates ten unique waypoints round-robin across the robots, with a goal tolerance of 0.15 m. Each waypoint ID belongs to exactly one robot, so no two robots visit the same point. If a robot times out or is blocked, its unfinished waypoint can be requeued to another robot. Completed waypoints are never reassigned. Table 3 gives the initial allocation.

Navigation: A*, path following and LiDAR guard. The navigation module has three parts: a custom A* grid planner, odometry-based path following and a forward LiDAR stop/resume guard. The planner reads the static collision boxes from the Gazebo SDF world. It works on a 0.10 m grid with 0.22 m obstacle inflation and shortens routes with line-of-sight checks. The guard stops the robot when forward returns fall within 0.45 m and lets it resume once clearance exceeds 0.60 m. In a dedicated test across `block_b`, the planner produced a four-point route. Its maximum lateral deviation from the straight start–goal segment is 0.70 m, which is a geometric property of the planned detour.

Nominal mission result (Run A). Run A is the corrected-world integration run of 18 October 2026, with the A* planner enabled and the boundary walls and static obstacles present in Gazebo. The executor reported `MISSION_COMPLETE`: wp01–wp10 were completed, and the unresolved and failed-robot lists were empty. Final goal distances were about 0.048–0.050 m, well inside the 0.15 m tolerance. The whole mission took 197.4 s from assignment. Robot r1 finished its three goals first (110.5 s), r2 followed at 147.7 s, and r0, which owns four goals, finished last (Figure 4). The log holds four `OBSTACLE_STOP`/`OBSTACLE_CLEARED` pairs (during wp03, wp04 and twice during wp07), so the stop/resume guard acted during the mission.

An earlier integration run, made before the A* planner, followed waypoints by direct odometry (goal distances 0.143–0.149 m) and also reached `MISSION_COMPLETE`. It is a different configuration, so I do not combine it with Run A.

(a) Supplied `view_frames` output (raster, full resolution)



(b) Legible redraw of the same frame tree (links below `base_link` are static transforms)

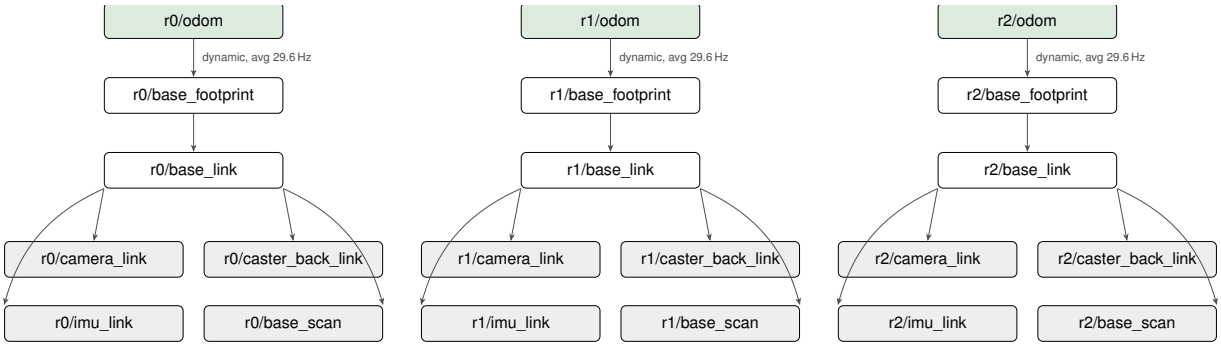
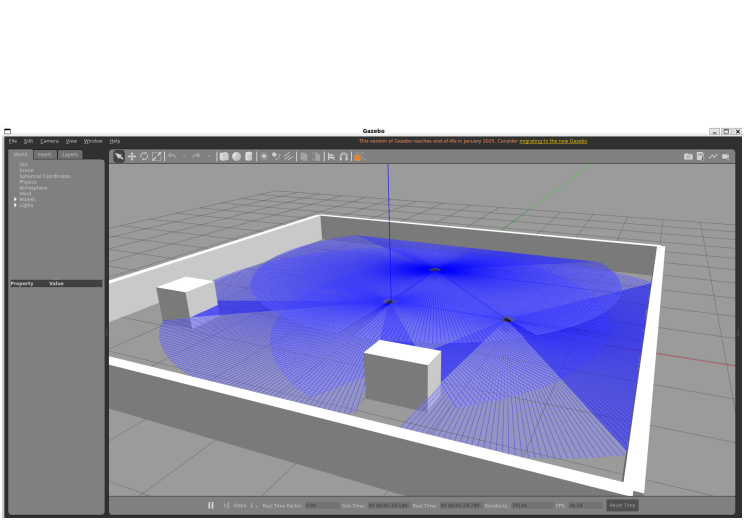
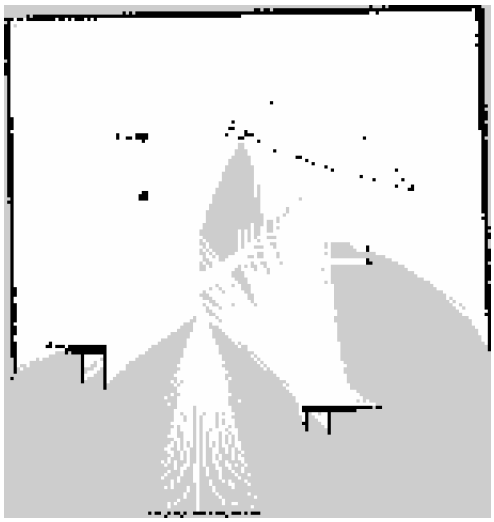


Figure 2. TF frame hierarchy of the three simulated robots, with robot-specific odometry, base and sensor frames. Panel (a) is the recorded `view_frames` output. Panel (b) redraws the same frames so they are readable at print size.



(a) Gazebo world, three robots, LiDAR rays



(b) SLAM occupancy map

Figure 3. (a) Gazebo Classic view: boundary walls, two boxes and three robots (small dark bodies). Blue surfaces are visualised LiDAR rays (sim time 00:01:19, real-time factor 0.86). (b) SLAM Toolbox occupancy map built from r0's LiDAR and odometry: 152 × 160 cells at 0.05 m, 63.5% known (image enlarged for display).

Failure detection and waypoint reassignment (separate experiment). A separate failure-injection run (`failure_recovery.log`) tested the software-level failure interface. I published a failure for r0 on `/mission/failed_robot`. The executor logged `ROBOT_FAILED` 8.3 s after assignment and redistributed r0's four unfinished waypoints (`REASSIGNED`: "Redistributed 4 unfinished waypoint(s)"). The remaining robots kept working with the same A*/guard pipeline, and r2 reached wp06 and wp09 with goal distances of 0.048–0.049 m. This run did not finish the whole mission (details in Section 3), so I report it on its own and do not merge it with Run A. It shows failure detection and goal reassignment working as implemented mechanisms.

When a robot stops responding. In my executor, a failure arrives as an explicit message, which makes the reassignment logic easy to test. In the field, a robot that hangs, loses its link or runs out of battery says nothing. The team has to infer the failure from missing heartbeats, so the timeout becomes a design choice. A short timeout may declare a healthy robot behind a weak link failed and send two robots to the same point. A

Table 3. Initial allocation of the ten waypoints (coordinates in metres, from the executor log).

Robot	Initial waypoints
r0	wp01 $(1,1)$, wp04 $(3,2)$, wp07 $(-1,3)$, wp10 $(-1,-1)$
r1	wp02 $(2,1)$, wp05 $(2,3)$, wp08 $(-2,2)$
r2	wp03 $(3,1)$, wp06 $(1,3)$, wp09 $(-2,1)$

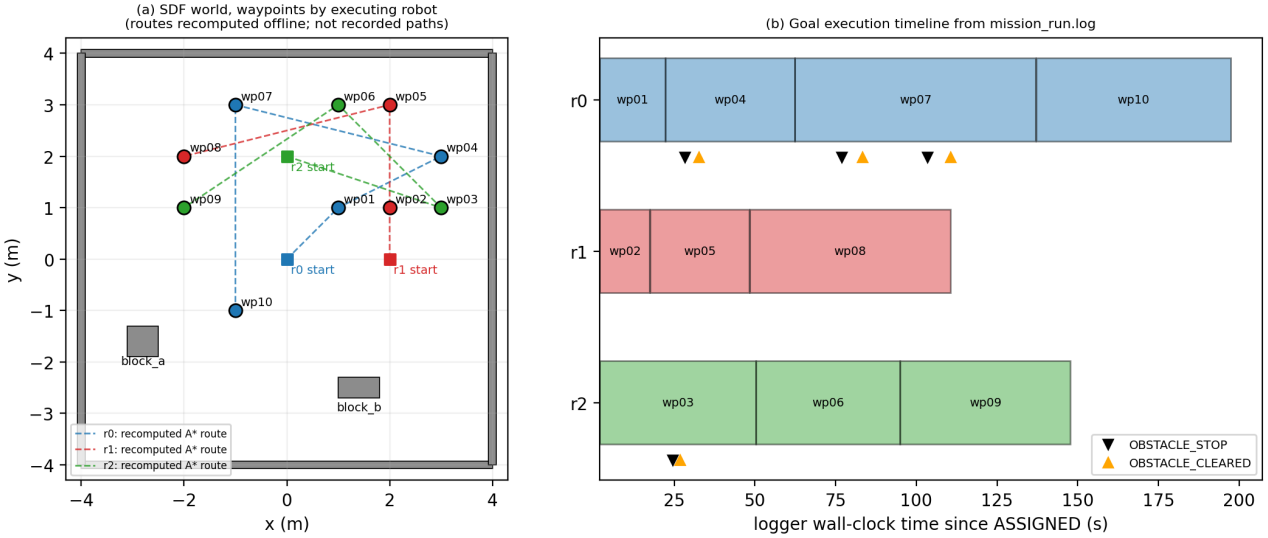


Figure 4. Run A (nominal mission, A* enabled). (a) Arena, boxes and waypoints, coloured by executing robot. Dashed lines are A* routes recomputed offline; they are not recorded trajectories. (b) Goal execution timeline from `mission_run.log` (wall-clock seconds since assignment), with obstacle-stop and clearance markers.

long timeout leaves the lost robot's goals unserved. A stopped robot also remains a physical obstacle that the others must plan around.

The simulator also gives us things that field systems lack: ground-truth-like odometry with little slip or drift, perfect time synchronisation, lossless local networking, a static world, and sensors that fail only when told to. These observations motivate the lease-based ownership and capability monitoring in Section 2.4.

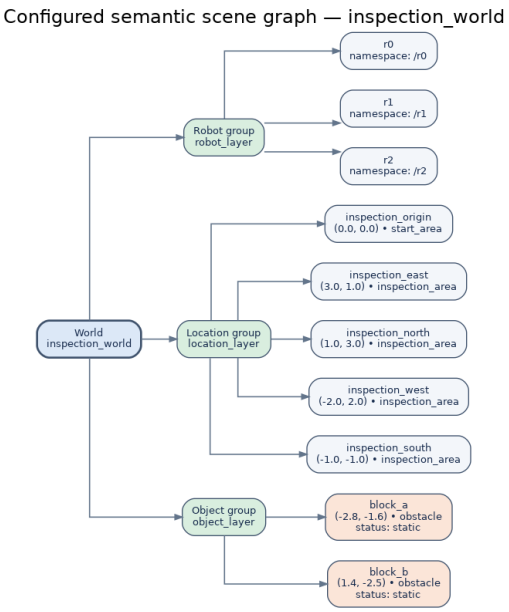
2.2 Language-Grounded Semantic Planning and Event Handling

Scene representation and grounding. The scene graph in `config/semantic/scene_graph.json` holds the inspection world, three robots, five named locations (`inspection_origin`, `_east`, `_north`, `_west`, `_south`), two static obstacles and their relations (Figure 5). It is configured world knowledge, not a scene built from live perception. Every node has metric attributes (location or obstacle-centre coordinates) and semantic attributes (entity type, obstacle status, robot namespace and relations).

The hierarchy runs from the world root to category groups (robot, location, object) and then to the configured entities. The function `validate_scene_hierarchy()` checks the root, the groups, the entity references, duplicate references, and whether every configured entity is covered. Tests cover a valid hierarchy and the rejection of a dangling location reference. The layered structure follows the spirit of Hydra and HOV-SG [1,2,3].

The planner resolves language to scene-graph nodes in two ways. The deterministic baseline in `semantic_planner.py` grounds location IDs and underscore-separated names that the request mentions. It chooses `inspect`, `observe` or `navigate` by keyword rules and gives tasks to available robots round-robin. It is a reproducible reference for the model-based planner. The model-based planner restricts the language model to the closed vocabulary of the scene graph, as described next.

Foundation-model planning. `llm_semantic_planner.py` uses the local Ollama model tag `qwen2.5:1.5b`. It relies on prompt-based structured generation with JSON output mode, non-streaming requests and temperature 0. There is no fine-tuning. I chose a small model so that the whole pipeline runs locally. The prompt allows only the actions `inspect`, `observe` and `navigate`, lists the configured location and robot IDs, forbids invented targets, and asks for an empty task list when the request is ambiguous or names an unknown target. The template is below; the brace-delimited items are filled in from the scene graph.



Group	Configured entities and attributes
Robot	r0, r1, r2; namespaces `/r0`, `/r1`, `/r2`
Location	`inspection_origin` $(0,0)$, `inspection_east` $(3,1)$, `inspection_north` $(1,3)$, `inspection_west` $(-2,2)$, `inspection_south` $(-1,-1)$; types `start_area` / `inspection_area`
Object	`block_a` $(-2.8,-1.6)$, `block_b` $(1.4,-2.5)$; type `obstacle`, status `static`

Figure 5. Configured semantic scene graph of `inspection_world` (left) and its metric and semantic attributes (right; coordinates in metres).

Table 4. Scripted events and resulting plan updates (CLI demonstration). The two events were run separately, each from the same starting plan (`inspection_east` → r0, `inspection_north` → r1); they are not consecutive.

Run	Event	State update	Resulting plan
Run 1	`robot_unavailable` r0	r0 excluded from assignment	`inspection_east` → r1; `inspection_north` → r2
Run 2	`target_unavailable` `inspection_east`	Target marked blocked; removed from the location collection and its hierarchy group	`inspection_east` blocked, not counted as completed; `inspection_north` stays with r1

```
You convert a robot mission into a JSON task plan.
Return JSON only, with this exact structure:
{"tasks":[{"task_id":"task_001","action":"inspect","target":"inspection_east","robot":"r0"}]}
Allowed actions: inspect, observe, navigate.
Use only these exact location IDs: {configured location IDs}
Use only these robot IDs: {configured robot IDs}
Do not invent targets. If the request is ambiguous or no target is known, return {"tasks":[]}
Mission: {natural-language mission request}
```

Ambiguity handling. The model abstains when a request is ambiguous: it returns an empty task list. The full planner then marks the plan invalid instead of passing the request to the deterministic fallback, and a regression test covers this. The fallback is used only when the request or response parsing raises an exception, and it is validated separately.

Plan validation. The validator checks whether the plan has the right structure, whether the task list is non-empty, whether task IDs are unique, and whether each action is supported. It checks whether every target is known, and whether every robot is known and available. When a hierarchy is present, it also validates it. Fields of the wrong type are rejected, which avoids set-membership type errors.

In the negative-case demonstration, the validator rejected a plan that targeted `unknown_place` (`valid = false`) with the error `tasks[0]: unknown target 'unknown_place'`. This plan would otherwise have been dispatched. Regression tests also cover invalid plans, alias grounding, invented targets, unavailable robots, an empty model response to an ambiguous request, and unsupported event types.

Event-driven replanning. `semantic_events.py` applies two scripted events (Table 4). A robot marked unavailable is excluded from later assignments. A target marked unavailable is recorded as blocked and removed from the active location collection and its hierarchy group. Grounding and hierarchy validation therefore stay consistent, and blocked work is never counted as completed. In the CLI demonstration (`event_replanning_demo.log`), both resulting plans were valid (`last_valid: true`) with no validation errors.

Verification results. The `colcon test` run recorded in the repository README collected 61 tests: 60 passed,

1 was skipped and none failed, with two non-fatal lint deprecation warnings. A separate run of the 58 non-lint tests of the evaluated commit also passed (the three ament lint tests need ROS and were not re-run there). Test counts depend on the commit, and the command in Section 5 regenerates the current numbers.

2.3 Long-Term Deployment Challenges

This section is literature-based analysis. It contains no experiments.

Lessons from field deployments. The two long-deployment references supplied with the assignment cover very different regimes. AutoInspect [4] is a ROS-based, mission-level autonomy system for inspection on a Spot robot. It covers mapping, localisation, graph-based navigation, mission execution and scheduling. Its authors report two deployments: 49 days at a robotics test facility (14 days of uninterrupted autonomous operation) and 35 days in the torus hall of the JET fusion reactor (15 uninterrupted days). They describe the second as the first fully autonomous mobile-robot deployment in a fusion facility. The system relies on an industrial-grade mesh network and docking-based charging. CERBERUS [5] is a heterogeneous team of legged and aerial robots that won the DARPA Subterranean Challenge final. It faced geometrically complex tunnels, degraded perception, no GNSS and denied communications. A single human supervisor commanded the team, and infrastructure such as a tethered rover acted as a communication hub. Its runs last about an hour.

Together they show a split. AutoInspect addresses duration for one robot. CERBERUS addresses team coordination under poor communication for short missions. A team whose task assignments rest on semantic knowledge that slowly becomes outdated sits between the two, and that is where the problem of Section 2.4 lies.

What changes over weeks and months. Four kinds of change need different responses.

- *Slow degradation of the robot.* Wheel and encoder wear, calibration drift, battery fade, dirt on a LiDAR window or lens, and thermal effects change what a robot can do without any single event. The change hides in streams that look normal: scans keep arriving with fewer valid returns, and odometry disagrees more with the IMU and with scan matching. Natural tools are monitors on residuals (predicted versus observed motion, observed versus map-predicted ranges, dropout statistics) with a change-point detector, and a capability estimate that can say "partly degraded". Ageing also changes the energy model, so route-length assumptions made at commissioning need updating.
- *Appearance change.* Lighting, season and cleaning schedules change how places look while the geometry stays the same. Place recognition must cope with this without treating every appearance change as a structural change.
- *Morphological evolution of the environment.* Walls, doors, racks and machinery are added or moved, temporary obstacles appear, and routes become narrow or blocked. This affects three layers separately: the geometric map, the semantic scene graph (a target is no longer where it was recorded) and the task preconditions (a target exists but a given robot cannot reach it). Khronos [6] makes the central point that long-term changes happen while the robot is not looking, so not having observed a change is not evidence that none occurred.
- *Degradation of information and coordination.* Communication loss makes a healthy robot look failed and leaves others acting on old state. A wrong inter-robot loop closure can merge maps incorrectly unless outliers are rejected [7]. A false change report can spread into a shared graph. Failures are correlated through shared firmware, dust and battery batches. A robot with low charge may be unable to take over a transferred task. Long-term SLAM surveys list map maintenance and long-run consistency among the open problems [8].

How much authority foundation models should have. My position is that a language model should propose and not decide. Table 5 gives the decision rights of each component in such an architecture.

A plan must pass four questions. *Grounding:* does every target exist in the scene? *Feasibility:* is there a capable robot? *Validity:* is the evidence behind the plan still current and reliable? *Execution safety:* can the task be done safely under present conditions? The validator of Section 2.2 covers grounding and robot availability. Extending it to evidence validity and execution safety is the focus of Section 2.4.

Three kinds of uncertainty need different responses. *Ambiguity* of a request is semantic ("check the east side" may match several nodes), and the right response is to ask. *Aleatoric* uncertainty [9] is randomness in the observations, such as sensor noise or occlusion, and is handled by probabilistic fusion and thresholds. *Epistemic* uncertainty is lack of knowledge that can be reduced, such as an unrefreshed scene graph or a model that has not seen the site, and is handled by re-observing, abstaining or escalating. A model's stated

Table 5. Separation of decision rights between model, checks and humans.

Component	Role	Authority
Language model	Interprets a request and proposes structured tasks	None over truth or safety; its output is a proposal
Validator	Grounding, feasibility and evidence-validity checks	Can reject a plan; cannot create one
Allocator	Chooses among validated tasks (energy, cost, capability)	Assigns work to robots
Runtime monitor	Watches execution independently of model and allocator	Can stop a robot and force replanning
Human supervisor	Resolves escalations and approves higher-risk tasks	Final

Table 6. Link between the proposed research and the four pillars of the call.

Pillar	Proposal
Robust perception in degraded conditions	Monitor sensor health; separate sudden failure, gradual degradation and uncertain observation
Scene representations that carry uncertainty	Attach provenance, observation time, confidence and a change rate to scene entities
Decision-making under uncertainty	Assign, re-observe, defer or escalate depending on evidence and capability
Agentic mission reasoning with foundation models	The model proposes structured tasks; deterministic checks and the execution layer keep authority

confidence is not a calibrated probability. KnowNo [10] uses conformal prediction so that a planner asks for help when its prediction set is not a single option. Its coverage guarantee needs calibration data that is exchangeable with deployment data, so on sites that change over months, recalibration becomes part of operations. SayCan [11] grounds proposals in robot affordances, and SayPlan [12] plans over a 3D scene graph with iterative replanning. Both strengthen feasibility and grounding.

In practice, failure modes can be bounded in six ways: (i) constrain the output to a schema and a closed vocabulary (already done here with JSON mode and fixed action names); (ii) check every plan against the current world model with deterministic code; (iii) keep an independent runtime monitor with override authority; (iv) ask the model to abstain or ask when its uncertainty set is not a single answer; (v) run new behaviour in shadow mode, with proposals logged against operator decisions, before it gets authority; (vi) pin the model and prompt under a regression suite so silent changes are detected. Authority should rise with reversibility and low consequence. Re-inspecting a known, accessible location after a validator pass can be automatic. Moving a robot into an area flagged as uncertain should involve a person.

Vision and the problem to attack. Maps are mostly built once and then queried. The next step is representations that carry a model of their own staleness and that task allocation actually uses. Each node would record when and by whom it was observed, how likely it is to have changed since, and what depends on it. The single problem I most want to address in the PhD is how a team keeps its task assignments consistent with a changing world when its evidence is old, partial and unevenly shared (Section 2.4). Table 6 links the proposal to the four pillars of the call.

2.4 Open Research Problem and Proposed Approach

This section is proposed research. It describes a framework and an evaluation plan, and it reports no implementation results.

Current state of long-term inspection autonomy. The practical state of the art is long but supervised operation of a single robot over a graph of known inspection points [4]. Teams of robots in harsh settings have been shown for short missions, with a human supervisor and communication infrastructure [5]. Scene representations have become hierarchical and open-vocabulary [1, 2, 3, 13], and recent work keeps them consistent as scenes change [6, 14]. Language-model planners ground their output in scene graphs and affordances [11, 12], use environment feedback [15] and ask for help when uncertain [10]. For team task allocation there is a taxonomy [16], capability-resilient and energy-aware formulations [17, 18], and belief-based reasoning under lost communication [19]. Table 7 places these lines of work next to the open space (I characterise each paper from its stated problem, method and contribution).

Table 7. Closest related work and the open space for the proposed problem.

Work	Covers	Open for this problem
AutoInspect [4]	Mapping, localisation, navigation and scheduling for long inspection deployments	Single-robot report; team reassignment; ageing of semantic evidence
CERBERUS [5]	Heterogeneous team under degraded perception and denied communication	Weeks-long operation; evidence validity of tasks
Khronos [6], 3D VSG [14]	Short- and long-term change in metric-semantic maps; per-object change likelihood	Using map change to decide whether a task may still be assigned
SayPlan [12], SayCan [11], Inner Monologue [15]	Language planning grounded in scene graph, affordances or feedback	Multi-agent use; scene knowledge that ages; unreliable feedback
KnowNo [10]	Calibrated uncertainty and requests for help	Calibration under distribution shift; team ownership
Mayya et al. [17], Notomista et al. [18]	Capability-aware, resilient, energy-aware allocation	Evidence validity of the target; network partitions
Bramblett and Bezzo [19]	Beliefs about teammates and assignments under lost communication	Beliefs about semantic targets going stale

Table 8. Literature assumptions versus deployment conditions.

Assumption	Real-world condition	Consequence
The scene graph given to the planner is true [12]	Doors, stock and equipment move, often unobserved [6]	A plan consistent with the graph but wrong about the world
Success feedback is reliable [15]	The detector's sensor is itself degrading	A mission "succeeds" on bad evidence
Capabilities are known or failures are announced [17,18]	Capability drifts, and is seen only through noisy health indicators	A half-blind robot that looks healthy gets the task
The planner sees global state [16]	Partitions, delays, a healthy robot that looks failed	Duplicate or orphaned tasks
The world model is correct [20]	Model error grows with time since commissioning	Overconfident beliefs and thresholds

Time-varying capability, human escalation, language-model grounding and allocation under lost communication have each been studied. The open question is how they interact: evidence validity, robot capability and task ownership during disconnection. This is the contribution I propose to study.

How real-world conditions affect the literature's assumptions. Table 8 lists assumptions made in the literature and how deployment conditions challenge them.

Several robots in one environment add more structure. Robots hold different beliefs after a partition and must reconcile them. One robot's wrong observation can enter the shared graph. Failures are correlated through shared hardware and software. Reassignment risks two robots executing the same task, and a stopped robot remains an obstacle. Bandwidth limits how much map-change evidence can be exchanged. The cost of a wrong decision grows with the number of robots acting on it.

Research question (proposed). *Under intermittent communication and time-varying robot capabilities, can a belief-aware task-validation and reassignment layer reduce stale-target executions and conflicting or duplicate work, compared with reactive and capability-only reassignment, while keeping mission completion and safety within predefined limits, at acceptable cost in energy, communication, recovery time and human escalations?*

The layer links each inspection task to timestamped, provenance-tagged semantic and spatial evidence. It enforces capability requirements and uses time-limited, locally enforced task ownership. The language model only proposes tasks (Figure 6).

I considered three alternatives and set them aside. A fixed staleness timeout is simple, but targets change at very different rates, so the formulation uses a per-target change rate. A purely symbolic planner would remove model errors, but it would also remove the free-text mission interface, so I keep the model and

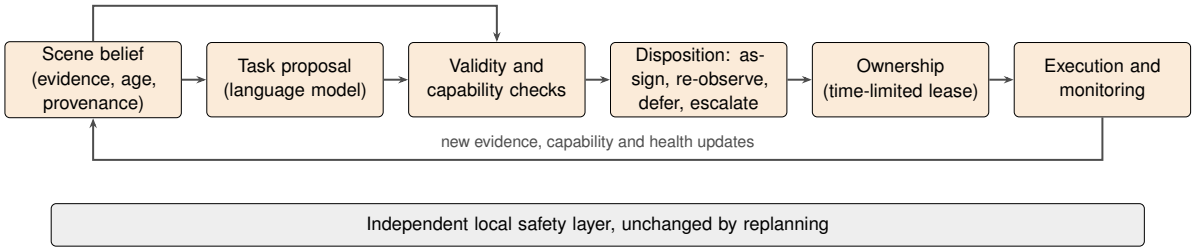


Figure 6. Proposed research framework (conceptual design, not implemented). The language model acts only at the proposal step. Validity checks, allocation and ownership are deterministic.

constrain it. A fully distributed allocator would add consensus and bandwidth effects that hide the effect I want to measure, so a central allocator comes first.

Formulation (proposed). Let T be the set of inspection tasks and R the set of robots, with decisions at times t in a rolling horizon. The exact problem is partially observable, because target states, robot health and teammate states are hidden. It is a POMDP [20]. I use belief summaries and solve a small decision problem in each cycle instead.

Evidence and validity. For task τ, let β_τ^0 be the belief, at the last observation time t_τ, that the target exists and can be inspected as described. It comes from a calibrated detector confidence, discounted by the health of the sensor that produced it. With a change rate $\lambda_\tau \geq 0$ for that kind of target, in the spirit of the variability attribute of [14],

$$\beta_\tau(t) = \beta_\tau^0 \, e^{-\lambda_\tau (t - t_\tau)}, \qquad v_\tau(t) = \mathbf{1}[\beta_\tau(t) \geq \beta_{\min,\tau}]\,. \tag{1}$$

The exponential form is a modelling assumption that needs one parameter per target. I would estimate λ_τ from data on how often each target type changes. Age and confidence combine, so a recent but uncertain observation and an old but confident one can both fall below $\beta_{\min,\tau}$.

Capability. For robot r and modality m, $\hat{c}_{r,m}(t) \in [0,1]$ is a health-based capability estimate. Task τ requires modalities M_τ at levels $\kappa_{\tau,m}$, so $c_{r\tau}(t) = \mathbf{1}[\hat{c}_{r,m}(t) \geq \kappa_{\tau,m} \ \forall m \in M_\tau]$. Let $c_{r\tau}^{\text{obs}}(t) \in \{0,1\}$ say whether r can observe the target of τ, and let $p_{r\tau} \in [0,1]$ be the estimated probability that r completes τ correctly, so a degraded robot can still be partly useful. The other inputs are: $\zeta_{\tau'}(t) \in \{0,1\}$ (task τ' observed complete); $a_r(t) \in \{0,1\}$ (r is reachable, holds a valid lease and is idle); $e_{r\tau}$ (energy needed for the task); $E_r(t)$ and E_r^{res} (remaining energy and reserve); and $\bar{x}_{r\tau}$ (previous assignment). None of these is a decision variable.

Decisions. For each open task and cycle there are four binary variables: $x_{r\tau}$ (execute), $o_{r\tau}$ (re-observe), d_τ (defer) and h_τ (escalate to a human).

$$\sum_r (x_{r\tau} + o_{r\tau}) + d_\tau + h_\tau = 1 \qquad \forall \tau \text{ open} \tag{C1}$$

$$x_{r\tau} \leq v_\tau(t)\, c_{r\tau}(t) \qquad \forall r, \tau \tag{C2}$$

$$o_{r\tau} \leq c_{r\tau}^{\text{obs}}(t)\big(1 - v_\tau(t)\big) \qquad \forall r, \tau \tag{C3}$$

$$x_{r\tau} \leq \zeta_{\tau'}(t) \qquad \forall \tau' \in \text{Pre}(\tau) \tag{C4}$$

$$\sum_\tau (x_{r\tau} + o_{r\tau}) \leq a_r(t) \qquad \forall r \tag{C5}$$

$$\sum_\tau e_{r\tau}\,(x_{r\tau} + o_{r\tau}) \leq E_r(t) - E_r^{\text{res}} \qquad \forall r \tag{C6}$$

$$w_{r\tau} \geq x_{r\tau} - \bar{x}_{r\tau}, \quad w_{r\tau} \geq \bar{x}_{r\tau} - x_{r\tau}, \quad w_{r\tau} \geq 0 \qquad \forall r, \tau \tag{C7}$$

In words: (C1) gives every open task exactly one disposition per cycle, so a stale task never makes the problem infeasible. Invalid evidence leads to re-observation, deferral or escalation. (C2) allows execution only with valid evidence and a capable robot. (C3) allows re-observation only when the evidence is no longer valid and the robot can sense the target. (C4) ties precedence to realised state. (C5) limits a robot to one task per cycle, and to zero when it is unreachable or its lease has lapsed. (C6) keeps an energy reserve. (C7) linearises $|x_{r\tau} - \bar{x}_{r\tau}|$, so both gaining and losing a task count (a transfer costs 2μ). The objective is

$$\max \sum_\tau V_\tau \sum_r p_{r\tau} x_{r\tau} + \sum_\tau \nu_\tau \sum_r o_{r\tau} - \sum_{r,\tau} \gamma_{r\tau}\,(x_{r\tau} + o_{r\tau}) \\ - \mu \sum_{r,\tau} w_{r\tau} - \rho_d \sum_\tau V_\tau d_\tau - \rho_h \sum_\tau V_\tau h_\tau, \tag{2}$$

where V_τ is the task value, $\gamma_{r\tau}$ is the travel and execution cost, and $\nu_\tau \approx V_\tau \cdot \Pr[\text{target still valid}]$ is the value of resolving uncertainty. Re-observation is an information-gathering action with a budget, as in [21]. In the

taxonomy of [16], this is a single-task, single-robot, instantaneous-assignment problem, which is polynomial without side constraints. With (C4) and (C6) it becomes a small mixed-integer program that suits teams of a few robots and need not be solved to optimality. After a re-observation, β_τ^0, t_τ and λ_τ are updated from the new evidence.

Task ownership during partitions (proposed protocol). A version number alone does not stop an isolated robot from continuing an old task, so ownership uses leases [22]:

1. An ownership record is (τ, r, k, L) with version k and lease duration L. A duration (not an absolute expiry time) makes the protocol less sensitive to clock offsets. The robot starts a monotonic local timer when it receives the record, and heartbeats renew it.
2. *Robot rule:* if the lease is not renewed in time, the robot stops the task and enters a defined safe state (hold, or return to dock).
3. *Planner rule:* after losing contact with r, the planner waits $L + \delta_{\text{msg}} + \delta_{\text{clk}}$ before reassigning, unless r acknowledged a release. Here δ_{msg} bounds message delay and δ_{clk} bounds clock-rate error over L. If these bounds cannot be trusted, the planner escalates.
4. Reassignment creates version $k + 1$, and the execution layer rejects older-version commands and results. A returning robot's old-version results are quarantined and reviewed, then used as timestamped evidence but not counted as completion.
5. A robot with an expired lease is still a physical obstacle. The others keep clearance around its last known position, with an uncertainty radius that grows with time, and the local safety layer stays active.

Under bounded message delay, bounded clock-rate error and a robot that follows the stop rule, no two robots hold a valid lease on the same task. The cost is availability: about L plus the grace term of idle time per lost robot. Shorter leases mean more heartbeat traffic and more false expiries.

Evaluation plan (proposed). The test bed would be the simulator of Section 2.1 and the scene graph of Section 2.2, extended with a simulated evidence model. The methods compared are: **B0**, static assignment with no replanning; **B1**, reactive reassignment on explicit failure or heartbeat timeout (close to the current executor); **B2**, capability-aware reassignment in the style of [17, 18]; **B3**, the proposed method (validity gate, capability monitor, re-observation, leases). Ablations of B3 remove, one at a time, the validity gate, the capability monitor, the leases, the hazard-based validity (replaced by a fixed timeout) and re-observation (defer only).

The scenarios are: **S1** abrupt robot failure; **S2** a target relocated or blocked and not yet seen; **S3** gradual sensor degradation, kept separate from S1 so that "failed" and "degraded but partly useful" can be told apart; **S4** a stale scene graph at the start, with 0, 10, 20 and 30% of targets changed; **S5** loss of communication to one robot, with randomised duration, delay and packet loss; **S6** combinations.

Scenarios are paired: a seed gives the same disturbance schedule to every method. The number of seeds comes from a pilot of about ten runs, chosen to reach a confidence-interval precision fixed in advance for the paired difference, with a floor of 30 seeds. Reporting uses bootstrap confidence intervals and a paired non-parametric test across all scenarios, including cases where B3 does not help. Simulator ground truth scores runs offline and is never visible to the planner.

Primary outcomes: **P1**, stale-target executions per mission (a task executed whose target is invalid in ground truth); **P2**, conflicting or duplicate executions per mission (including execution under an expired lease). *Secondary outcomes:* fraction of required tasks completed correctly; fraction of missions fully completed; recovery time; extra travel and energy; messages and bytes; human escalations per mission. Safety is reported separately: collisions, unsafe-clearance events and assignments that fail a required check.

H1. B3 lowers P1 and P2 relative to B1 and B2 (paired comparison).

H2. The task-completion fraction of B3 is not worse than that of B1 by more than $\delta_c = 5$ percentage points (non-inferiority).

H3. B3 escalates no more than $\rho_{\max} = 10\%$ of tasks (an assumed operator budget of a few interventions per mission).

H4. Extra communication and idle time are reported as costs, with no target set.

The values δ_c and $\rho_{\max}$ are design choices to be fixed before the main experiments. The hypotheses can be rejected: for example, if the validity gate triggers so many re-observations that completion falls below the margin.

Expected contribution. A testable framework that couples evidence validity, capability monitoring and task ownership, so that task assignments stay consistent with robot capabilities, scene evidence and a changing environment. Experiments would show whether this coupling reduces stale-target executions and duplicate work compared with simpler reassignment. The implemented testbed of Sections 2.1–2.2 supplies the simulator, scene graph, validator and failure interface on which the study would build.

3. Limitations and Future Work

This section collects the boundaries of the current implementation and evaluation, with the next step for each.

3.1 Navigation and mapping

- *Planning source.* The custom A* planner uses static Gazebo SDF geometry, not the saved SLAM map. The LiDAR guard stops and resumes the robot but does not reroute around dynamic obstacles, so collision-free motion is not guaranteed. *Next:* plan on the SLAM occupancy grid with an inflated costmap, and add local replanning around dynamic obstacles.
- *Nav2.* Nav2 global planning and control were not configured or validated in the demonstrated mission. *Next:* configure a namespaced Nav2 stack and compare it with the custom pipeline on the same ten-waypoint mission.
- *Mapping.* The SLAM result is a partial map from r0 alone (36.5% unknown). It has no ground-truth accuracy evaluation and no multi-robot map merging. *Next:* full-coverage exploration, a comparison against the SDF ground truth, and map merging across the three robots.
- *Failure recovery.* In the recorded failure-injection log (34 lines), the team did not complete the mission after reassignment. Robot r2 reached wp06 and wp09 and then started wp04. Robot r1 logged seven obstacle stops at 0.18–0.40 m and timed out on wp02, wp03 and wp05 after about 91 s each, with requeues. I did not diagnose the cause. Mutual blocking among the surviving robots and the stopped r0 is a plausible but untested explanation, because the stop-only guard and the static-geometry planner cannot resolve it. The failure was injected as a message, not as a physical fault. *Next:* log inter-robot distances, add stopped-robot avoidance and inter-robot coordination, and repeat the experiment over several seeds.
- *Sensor-rate measurements.* The rates are short-run observations under WSL2 load, not controlled benchmarks. *Next:* longer logged measurements at a controlled real-time factor.

3.2 Semantic planning

- *Scene knowledge.* The scene graph holds configured world knowledge, not automatically perceived scenes. Its hierarchy is organised by entity category and has no spatial layer (places, rooms) like Hydra or HOV-SG [1,2,3]. *Next:* populate the graph from perception, and add containment and traversability relations.
- *Language understanding.* The deterministic baseline relies on ID and keyword grounding. The language-model planner abstains on ambiguity without asking a clarifying question. The mocked empty-response test checks response handling only, and the behaviour of a 1.5B-parameter model need not carry over to larger models. *Next:* a clarification dialogue, a benchmark set of missions, and a comparison across model sizes.
- *Execution integration.* Event handling works at software level and is not yet integrated into end-to-end robot navigation. The model does not regenerate plans after each event, and sensor-capability, task-dependency and physical-safety constraints are not enforced at this layer. *Next:* translate validated tasks into `mission_executor` goals, and connect semantic events to the executor's failure interface.

3.3 Long-term deployment

Evidence freshness over time, gradual sensor-health degradation, communication-resilient task ownership and physical deployment remain future research. They form the core of the proposal in Section 2.4. The first steps are an evidence-age model and capability monitors in the simulator, then the lease protocol and the paired evaluation, and finally trials on physical robots.

4. Conclusion

The project delivers a three-robot simulation baseline and a mapping and navigation pipeline that completed all ten unique waypoints in the recorded A*-enabled run. It adds a failure-detection and reassignment

mechanism, and a semantic planning pipeline with scene-graph grounding, structured language-model planning, validation and event-driven replanning, backed by a recorded 61-test regression run. These components lead to the proposal of Section 2.4: treat the evidence behind each inspection task as something that ages, check it together with robot capability, and keep task ownership safe when communication fails. The proposal is a hypothesis with an evaluation plan, and Section 3 sets out the concrete next steps.

5. Reproducibility

Code and evidence: `github.com/Akshit-Sh/ltu-robotics-ai-phd-assignment`. The evaluated commit is `268461f` (19 October 2026); after cloning, run `git checkout 268461f` to get that exact revision. Environment: Ubuntu 22.04 (WSL2), ROS 2 Humble, Gazebo Classic, TurtleBot3, `slam_toolbox`, and, for the model-backed planner, a local Ollama installation with `qwen2.5:1.5b` pulled. Sections 2.3 and 2.4 contain no code. The recorded logs in `docs/evidence/` are the source of the numbers and figures above. Fresh runs depend on timing, load and real-time factor, so they will not match the logs exactly.

```
source /opt/ros/humble/setup.bash
cd ~/ltu_rai_assignment/ros2_ws
colcon build --packages-select multi_robot_sim --symlink-install
source install/setup.bash
export TURTLEBOT3_MODEL=burger LIBGL_ALWAYS_SOFTWARE=1
ros2 launch multi_robot_sim multi_robot.launch.py    # terminal 1: simulation
ros2 run multi_robot_sim mission_executor            # terminal 2: expect MISSION_COMPLETE

# optional mapping (terminal 3), then save the map:
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file:=$HOME/ltu_rai_assignment/config/slam_r0.yaml \
  use_sim_time:=true
ros2 run nav2_map_server map_saver_cli -f maps/inspection_map

# failure injection (publish once, while r0 still has unfinished work):
ros2 topic pub --once /mission/failed_robot \
  std_msgs/msg/String "{data: r0}"

# semantic demonstrations and tests:
ros2 run multi_robot_sim semantic_planner
ros2 run multi_robot_sim llm_semantic_planner
ros2 run multi_robot_sim semantic_events
colcon test --packages-select multi_robot_sim && colcon test-result --verbose
```

A nominal run is verified by `MISSION_COMPLETE`, all ten waypoint IDs in the completed list, and empty unresolved and failed-robot lists. The failure test is verified by `ROBOT_FAILED` and a reassignment message. The committed map was produced with the SLAM configuration above; a fresh map will differ in detail from Figure 3(b).

6. Time Spent and AI Assistance Declaration

Approximate time per task. I did not track time as I went, so these are retrospective estimates: Task 2.1 (simulation, sensing, mapping and navigation), 14 h; Task 2.2 (semantic planning and event handling), 7 h; Task 2.3 (long-term deployment challenges), 3 h; Task 2.4 (open research problem and proposed approach), 5 h. The total is 29 h.

AI assistance and author verification. I used ChatGPT for troubleshooting, for reviewing test and evidence outputs, and for help with report organisation and editing. For Sections 2.3 and 2.4 I also used Claude (Anthropic) to search for and summarise related work, to critique drafts against the assignment text, and to help draft and tighten the wording. Claude was also used for the layout and wording revision of this report; the numbers and claims were carried over from the earlier versions and checked against the repository logs. I ran every command whose output is reported, and the experimental results in Sections 2.1 and 2.2 come from those runs. For the references added in the previous revision, I checked that each exists and that authors, venue and identifier match the arXiv, publisher or proceedings page. My descriptions of what these papers cover rest on their abstracts and introductions, and I did not reproduce their results.

References

[1] N. Hughes, Y. Chang, and L. Carlone. Hydra: A real-time spatial perception system for 3D scene graph construction and optimization. *Robotics: Science and Systems (RSS)*, 2022. doi:10.15607/RSS.2022.XVIII.050

[2] N. Hughes, Y. Chang, S. Hu, R. Talak, R. Abdulhai, J. Strader, and L. Carlone. Foundations of spatial perception for robotics: Hierarchical representations and real-time systems. *Int. J. Robotics Research*, 43(10):1457–1505, 2024. doi:10.1177/02783649241229725

[3] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard. Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation. *RSS*, 2024. doi:10.15607/RSS.2024.XX.070

[4] M. Staniaszek, T. Flatscher, J. Rowell, H. Niu, W. Liu, Y. You, R. Skilton, M. Fallon, and N. Hawes. AutoInspect: Towards long-term autonomous industrial inspection. *IEEE Trans. Field Robotics*, pp. 529–548, 2025 (earlier version: ICRA Workshop on Field Robotics 2024, arXiv:2404.12785). doi:10.1109/TFR.2025.3586831

[5] M. Tranzatto et al. CERBERUS in the DARPA Subterranean Challenge. *Science Robotics*, 7(66), 2022. doi:10.1126/scirobotics.abp9742

[6] L. Schmid, M. Abate, Y. Chang, and L. Carlone. Khronos: A unified approach for spatio-temporal metric-semantic SLAM in dynamic environments. *RSS*, 2024.

[7] Y. Tian et al. Kimera-Multi: Robust, distributed, dense metric-semantic SLAM for multi-robot systems. *IEEE Trans. Robotics*, 38(4):2022–2038, 2022. doi:10.1109/TRO.2021.3137751

[8] R. B. Sousa, H. M. Sobreira, and A. P. Moreira. A systematic literature review on long-term localization and mapping for mobile robots. *J. Field Robotics*, 40(5):1245–1322, 2023.

[9] E. Hüllermeier and W. Waegeman. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning*, 110(3):457–506, 2021. doi:10.1007/s10994-021-05946-3

[10] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, et al. Robots that ask for help: Uncertainty alignment for large language model planners. *CoRL*, PMLR 229, 2023.

[11] M. Ahn et al. Do as I can, not as I say: Grounding language in robotic affordances. *CoRL*, 2022. arXiv:2204.01691

[12] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. *CoRL*, PMLR 229:23–72, 2023. arXiv:2307.06135

[13] Q. Gu et al. ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning. *IEEE ICRA*, 2024. doi:10.1109/ICRA57147.2024.10610243

[14] S. Looper, J. Rodriguez-Puigvert, R. Siegwart, C. Cadena, and L. Schmid. 3D VSG: Long-term semantic scene change prediction through 3D variable scene graphs. *IEEE ICRA*, 2023. arXiv:2209.07896

[15] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, et al. Inner Monologue: Embodied reasoning through planning with language models. *CoRL*, PMLR 205:1769–1782, 2023.

[16] B. P. Gerkey and M. J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *Int. J. Robotics Research*, 23(9):939–954, 2004. doi:10.1177/0278364904045564

[17] S. Mayya, D. Salazar-D'Antonio, D. Saldaña, and V. Kumar. Resilient task allocation in heterogeneous multi-robot systems. *IEEE Robotics and Automation Letters*, 6(2):1327–1334, 2021.

[18] G. Notomista, S. Mayya, Y. Emam, C. Kroninger, A. Bohannon, S. Hutchinson, and M. Egerstedt. A resilient and energy-aware task allocation framework for heterogeneous multi-robot systems. 2021. arXiv:2105.05586

[19] L. Bramblett and N. Bezzo. Epistemic planning for multi-robot systems in communication-restricted environments. *Frontiers in Robotics and AI*, 10, 2023.

[20] M. Lauri, D. Hsu, and J. Pajarinen. Partially observable Markov decision processes in robotics: A survey. *IEEE Trans. Robotics*, 39(1):21–40, 2023. doi:10.1109/TRO.2022.3200138

[21] G. A. Hollinger and G. S. Sukhatme. Sampling-based robotic information gathering algorithms. *Int. J. Robotics Research*, 33(9):1271–1287, 2014. doi:10.1177/0278364914533443

[22] C. G. Gray and D. R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *Proc. 12th ACM Symp. Operating Systems Principles (SOSP)*, pp. 202–210, 1989.